

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc. I.T Semester V) Dot .Net
Core and Progressive Web Application Practical

Practical – III

Roll No.: T026	Name: Hamiza Imtiyaz Mujawar
Class: TYIT	Batch: 1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

a) Create methods add(), multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

Step 1 - Start.

Step 2 - Declare a delegate with two integer parameters.

Step 3 - Create methods:

Add(int a, int b)

Subtract(int a, int b)

Multiply(int a, int b)

Divide(int a, int b)

Step 4 - In the Main() method:

Create a delegate object.

Assign the Add() method to the delegate and call it.

Assign the Subtract() method to the delegate and call it.

Assign the Multiply() method to the delegate and call it.

Assign the Divide() method to the delegate and call it.

Step 5 - Display the results.

Step 6 - Stop.

Code:

```
using System;
```

```
delegate void operation(int a, int b);
```

```
class Calculator
```

```
{
```

```
public void add(int a, int b)
```

```
{
```

```
Console.WriteLine("Addition of number : " + (a + b));
```

```
}
```

```
public void sub(int a, int b)
```

```
{
```

```
Console.WriteLine("Subtraction of number : " + (a - b));
```

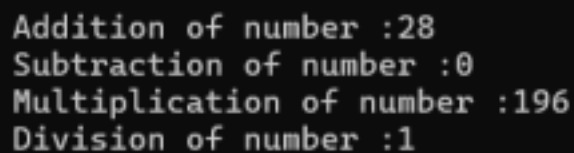
```
}  
public void mul(int a, int b)
```

Hamiza Imtiyaz Mujawar T026

```
{  
Console.WriteLine("Multiplication of number :" + (a * b));  
}  
public void div(int a, int b)  
{  
Console.WriteLine("Division of number :" + (a / b));  
}  
  
}
```

```
class Program  
{  
static void Main()  
{  
Calculator c = new Calculator();  
operation obj;  
  
obj = c.add;  
obj(14, 14);  
  
obj = c.sub;  
obj(14, 14);  
  
obj = c.mul;  
obj(14, 14);  
  
obj = c.div;  
obj(14, 14);  
  
}  
}
```

Output:



```
Addition of number :28  
Subtraction of number :0  
Multiplication of number :196  
Division of number :1
```

b) Write a program using multicast delegate.

Algorithm:

Step 1 - Start.

Step 2- Declare a delegate with no parameters.

Step 3 - Create three methods (e.g., Welcome(), Learn(), ThankYou()).

Step 4 - In the Main() method:

Create a delegate object.

Assign the first method to the delegate.

Add the remaining methods using the += operator.

Hamiza Imtiyaz Mujawar T026

Invoke the delegate object.

Step 5 - All methods execute one after another.

Step 6 - Stop.

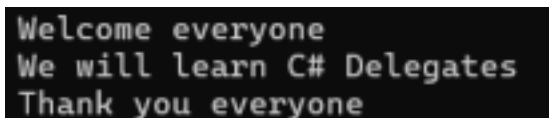
Code:

```
using System;
delegate void Message();
class Demo
{
    public void Welcome()

    {
        Console.WriteLine("Welcome everyone");
    }
    public void Learn()
    {
        Console.WriteLine("We will learn C# Delegates");
    }
    public void thanku()
    {
        Console.WriteLine("Thank you everyone");
    }
}

class Program
{
    static void Main()
    {
        Demo d = new Demo();
        Message msg;
        msg = d.Welcome;
        msg += d.Learn;
        msg += d.thanku;
        msg();
    }
}
```

Output:

A screenshot of a terminal window showing the output of the C# program. The text is displayed on three lines: "Welcome everyone", "We will learn C# Delegates", and "Thank you everyone". The background is dark, and the text is light-colored.

```
Welcome everyone
We will learn C# Delegates
Thank you everyone
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

Step 1 - Start.

Step 2 - Create a class BankAccount.

Step 3 - Declare private data members:

AccountNumber

Hamiza Imtiyaz Mujawar T026

Balance

Step 4 - Create a property for Balance.

Step 5 - In the set accessor:

Check whether the assigned value is greater than or equal to 0.

If yes, store the value in Balance.

Otherwise, display an error message: "Balance cannot be negative."

Step 6 - Create an object of the BankAccount class in the Main() method.

Step 7 - Assign the account number.

Step 8 - Assign a valid balance and display the account details.

Step 9 - If a negative balance is entered, display the validation message.

Step 10 - Stop.

Code:

```
using System;
```

```
class BankAcc
```

```
{  
    public int AccNumber { get; set; }
```

```
    private double balance;
```

```
    public double Balance
```

```
    {  
        get  
        {  
            return balance;  
        }  
    }
```

```
    set
```

```
    {  
        if (value < 0)  
        {  
            Console.WriteLine("Balance cannot be negative.");  
        }  
        else  
        {  
            balance = value;  
        }  
    }  
}
```

```
class Program
```

```
{  
    static void Main(string[] args)  
    {
```

Hamiza Imtiyaz Mujawar T026

```
        BankAcc account = new BankAcc();
```

```
        account.AccNumber = 345;  
        account.Balance = 1000;
```

```
        Console.WriteLine("Account Number: " + account.AccNumber);  
        Console.WriteLine("Balance: " + account.Balance);  
        // Attempt to set a negative balance  
        account.Balance = -500;
```

```
        Console.WriteLine("Balance after attempting to set negative value: " +  
            account.Balance);
```

```
    }  
}
```

Output:

```
Account Number: 345  
Balance: 1000  
Balance cannot be negative.  
Balance after attempting to set negative value: 1000
```